

RoomKey DB



Hecho por
José María
Gómez Vélez

Índice

1. Introducción.....	3
2. Modelo entidad relación.....	4
3. Modelo relacional en Workbench.....	5
4. Consultas SQL.....	6
5. Vistas.....	13
6. Funciones.....	15
7. Procedimientos.....	17
8. Triggers.....	20
Valoración personal.....	23

1. Introducción

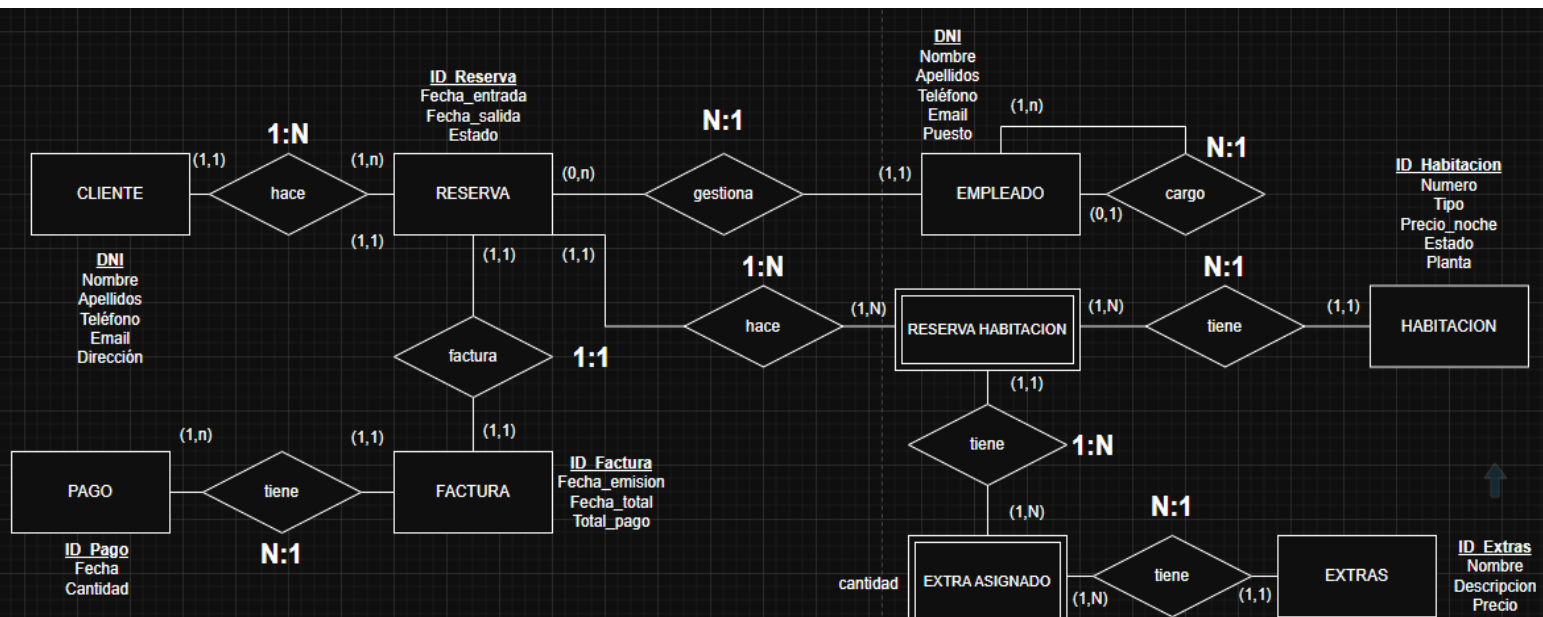
El objetivo principal de este proyecto es el **diseño y modelado** de una base de datos relacional para la gestión integral de un hotel. La finalidad es sistematizar y almacenar de forma eficiente la información relacionada con el funcionamiento diario del establecimiento.

El sistema permite administrar:

- Clientes: Registro de datos personales y de contacto.
- Reservas: Control de fechas, estados y asignación de habitaciones.
- Habitaciones: Gestión del inventario, tipos, precios y características extras.
- Facturación: Crear y gestionar las facturas propias.
- Empleados: Gestión del personal y su propia jerarquía.

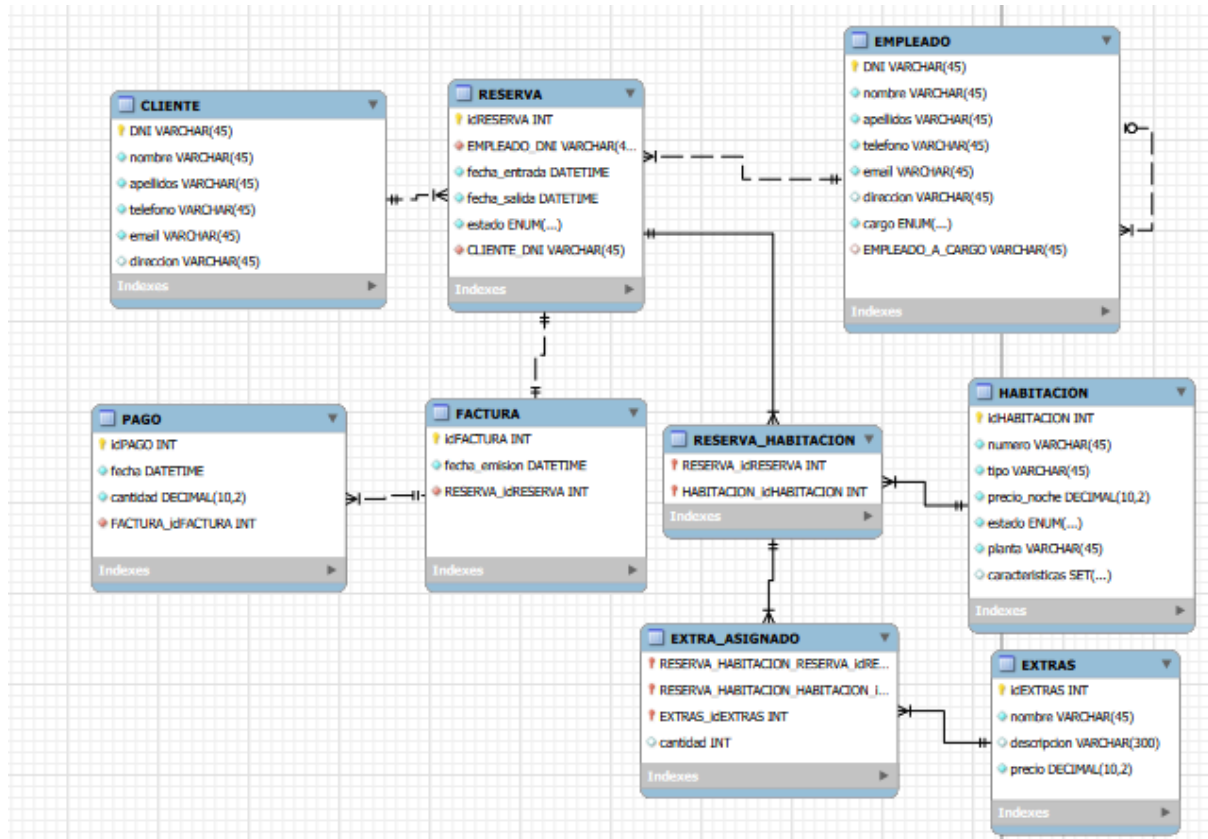
Este diseño busca resolver problemas comunes como la duplicidad de datos, la pérdida de información de reservas y facilitar la consultas del hotel

2. Modelo entidad relación



En esta fase nos centramos en ver a los principales participantes del hotel (Clientes, Empleados) y las cosas físicas (Habitaciones, Extras) para saber cómo interactúan entre sí. **La idea principal es que el Cliente realiza una Reserva, la cual es gestionada por un Empleado, esta genera una Factura y la Habitación reservada, puede tener incluir extras.**

3. Modelo relacional en Workbench



Aquí en el modelo relacional podemos ver de una mejor manera las claves primarias y foráneas, así como también se puede llegar a observar el uso del **ENUM()** y del **SET()**.

4. Consultas SQL

Obtener un listado detallado de los servicios adicionales solicitados por los clientes que acaban de realizar el **Check-in**.

```

select c.nombre, h.numero, h.tipo, e.nombre, ea.cantidad, r.estado
from CLIENTE c inner join RESERVA r
  on c.DNI = r.CLIENTE_DNI
inner join RESERVA_HABITACION rh
  on r.idRESERVA = rh.RESERVA_idRESERVA
inner join HABITACION h
  on rh.HABITACION_idHABITACION = h.idHABITACION
inner join EXTRA_ASIGNADO ea
  on r.idRESERVA = ea.RESERVA_HABITACION_RESERVA_idRESERVA
inner join EXTRAS e
  on ea.EXTRAS_idEXTRAS = e.idEXTRAS
where r.estado = "Check-in";

```

AZ nombre	AZ numero	AZ tipo	AZ nombre	123 cantidad	AZ estado
Ransell	366	Familiar	Cuna	1	Check-in
Ransell	198	Deluxe	Cuna	1	Check-in
Ransell	242	Suite	Cuna	1	Check-in
Peta	443	Deluxe	Cuna	1	Check-in
Peta	174	Familiar	Cuna	1	Check-in
Peta	250	Simple	Cuna	1	Check-in
Johannes	352	Doble	Cuna	1	Check-in
Johannes	130	Doble	Cuna	1	Check-in
Johannes	404	Simple	Cuna	1	Check-in
Johannes	489	Doble	Cuna	1	Check-in
Elfrida	352	Doble	Cuna	2	Check-in
Elfrida	301	Doble	Cuna	2	Check-in
Elfrida	110	Suite	Cuna	2	Check-in
Paddie	176	Doble	Cuna	1	Check-in
Paddie	295	Simple	Cuna	1	Check-in
Becky	331	Familiar	Cuna	2	Check-in
Jamie	328	Doble	Cuna	1	Check-in
Gwenneth	445	Doble	Cuna	1	Check-in
Gwenneth	296	Suite	Cuna	1	Check-in

Calcular el total de ingresos recaudados agrupados por **fecha**, para identificar los días de mayor actividad financiera en el hotel.

```
select DATE(p.fecha) as dia, COUNT(*) as num_transacciones, SUM(p.cantidad) as total_dia
from PAGO p
group by DATE(fecha)
order by dia;
```

dia	num_transacciones	total_dia
2024-01-10	3	122.17
2024-01-11	2	68.99
2024-01-12	1	37.89
2024-01-14	1	36.89
2024-01-15	1	93.62
2024-01-16	1	10.03
2024-01-18	1	19.31
2024-01-20	3	112.28
2024-01-21	1	13.56
2024-01-24	1	59.25
2024-01-25	2	105.32
2024-01-26	1	40.16
2024-01-27	5	177.72

Listar las habitaciones cuyo precio por noche es mayor al promedio del hotel, permitiendo identificar la oferta premium.

```
select h.numero, h.tipo, h.precio_noche
from HABITACION h
where h.precio_noche > (select AVG(h.precio_noche) from HABITACION h);
```

TACION 1 X

h.numero, h.tipo, h.precio_noche from HABITACION h where h.precio_noche > (select AVG(h.precio_noche) from HABITACION h) Enter a SQL expression to filter

AZ numero	AZ tipo	123 precio_noche
441	Suite	279.89
343	Doble	347.33
142	Doble	308.56
355	Familiar	342.12
248	Doble	188.85
326	Suite	304.86
429	Deluxe	255.21
107	Deluxe	308.19
275	Suite	212.21
434	Deluxe	319.93
425	Simple	329.33
251	Familiar	345.15
439	Simple	226.45

Determinar la duración total en días de cada reserva realizada por los clientes para la planificación de limpieza.

```

select r.CLIENTE_DNI, r.fecha_entrada, r.fecha_salida,
       DATEDIFF(r.fecha_salida, r.fecha_entrada) AS noches
from RESERVA r;

```

VA 1 X

CLIENTE_DNI, r.fecha_entrada, r.fecha_salida, DATEDIFF(r.fecha_salida, r.fecha_entrada) AS noches | Enter a SQL expression to filter results (use Ctrl)

AZ CLIENTE_DNI	fecha_entrada	fecha_salida	123 noches
85529063A	2025-02-26 16:46:55	2025-02-28 16:46:55	2
73801217A	2024-02-28 17:56:57	2024-03-01 17:56:57	2
26137378A	2024-10-17 01:01:40	2024-10-23 01:01:40	6
03452044A	2025-01-31 09:37:15	2025-02-06 09:37:15	6
36740274A	2025-01-26 17:32:50	2025-01-31 17:32:50	5
47278331A	2026-01-27 04:44:23	2026-01-30 04:44:23	3
51953172A	2025-02-10 22:29:34	2025-02-14 22:29:34	4
47702240A	2025-06-16 17:22:42	2025-06-21 17:22:42	5
73567991A	2024-07-03 07:17:55	2024-07-10 07:17:55	7
46306129A	2025-02-22 22:54:51	2025-02-25 22:54:51	3
22874855A	2025-02-20 12:13:48	2025-02-24 12:13:48	4
86264038A	2024-11-20 16:51:43	2024-11-21 16:51:43	1
25530512A	2025-04-06 04:23:11	2025-04-11 04:23:11	5
16794148A	2025-09-03 06:54:54	2025-09-08 06:54:54	5
82911840A	2025-04-04 20:12:50	2025-04-07 20:12:50	3

Generar un informe que muestre qué **empleado** gestionó cada **reserva** y la **factura** asociada, permitiendo auditar la actividad comercial de la plantilla.

```

select e.nombre AS nombre_empleado, e.cargo, r.idRESERVA, f.idFACTURA, f.fecha_emision
from EMPLEADO e inner join RESERVA r
  on e.DNI = r.EMPLEADO_DNI
inner join FACTURA f
  on r.idRESERVA = f.RESERVA_idRESERVA
order by e.cargo;

```

AZ nombre_empleado	AZ cargo	123 idRESERVA	123 idFACTURA	fecha_emision
Amble	Gerente	37	37	2026-02-06 11:43:30
Amble	Gerente	370	370	2026-02-06 11:43:30
Amble	Gerente	517	517	2026-02-06 11:43:30
Amble	Gerente	551	551	2026-02-06 11:43:30
Amble	Gerente	618	618	2026-02-06 11:43:30
Amble	Gerente	711	711	2025-04-02 22:04:20
Amble	Gerente	715	715	2024-07-29 07:09:04
Amble	Gerente	718	718	2026-02-06 11:43:30
Amble	Gerente	799	799	2025-05-09 06:17:32
Natasha	Gerente	219	219	2024-11-21 16:51:43
Natasha	Gerente	353	353	2026-01-05 18:01:48
Natasha	Gerente	368	368	2025-06-28 10:40:50
Natasha	Gerente	502	502	2025-12-31 09:59:07
Natasha	Gerente	524	524	2026-02-06 11:43:30
Natasha	Gerente	609	609	2024-12-02 00:04:28
Natasha	Gerente	752	752	2024-10-13 02:35:43
Natasha	Gerente	778	778	2024-04-18 03:46:09
Natasha	Gerente	792	792	2025-07-17 03:39:41
Natasha	Gerente	810	810	2026-02-06 11:43:30

Identificar todas las habitaciones que actualmente no están disponibles para su venta debido a que su estado es **'Sucia'** o **'Mantenimiento'**, facilitando la organización del equipo de limpieza.

```

select h.numero, h.tipo, h.planta, h.estado
from HABITACION h
where estado = 'Sucia' or estado = 'Mantenimiento';

```

TACION 1 X

h.numero, h.tipo, h.planta, h.estado from HABITACION h where Enter a SQL expression to filter results (u

	AZ numero ▼	AZ tipo ▼	AZ planta ▼	AZ estado ▼	
	441	Suite	3	Sucia	
	343	Doble	1	Mantenimiento	
	188	Familiar	4	Sucia	
	454	Deluxe	4	Mantenimiento	
	248	Doble	5	Sucia	
	429	Deluxe	1	Sucia	
	107	Deluxe	4	Mantenimiento	
	327	Simple	1	Mantenimiento	
	425	Simple	2	Sucia	
	212	Doble	1	Mantenimiento	
	300	Simple	2	Mantenimiento	
	182	Simple	2	Sucia	
	443	Deluxe	4	Mantenimiento	
	352	Doble	3	Sucia	
	272	Familiar	5	Sucia	
	241	Suite	3	Mantenimiento	
	433	Suite	1	Sucia	
	176	Doble	1	Sucia	
	331	Familiar	4	Sucia	

Listar los detalles de las facturas que han recibido pagos únicos superiores a **90€**, con el objetivo de identificar las transacciones de mayor impacto económico para el hotel.

```

SELECT f.idFACTURA, f.fecha_emision, p.cantidad, p.fecha AS fecha_pago
FROM FACTURA f
INNER JOIN PAGO p ON f.idFACTURA = p.FACTURA_idFACTURA
WHERE p.cantidad > 90;

```

JRA(+) 1 X

f.idFACTURA, f.fecha_emision, p.cantidad, p.fecha AS fecha_pago | Enter a SQL expression to filter results

	123 idFACTURA	fecha_emision	123 cantidad	fecha_pago
	917	2026-02-06 11:43:30	99.96	2024-09-15 11:27:30
	579	2026-02-06 11:43:30	93.52	2026-02-09 19:09:39
	851	2026-02-06 11:43:30	92.3	2025-01-30 16:28:24
	151	2025-03-25 23:46:49	95.21	2025-07-11 00:16:00
	745	2026-01-16 20:22:47	94.94	2024-06-29 13:01:21
	126	2025-08-03 20:00:10	97.93	2025-09-04 03:51:00
	494	2026-02-06 11:43:30	95.76	2024-08-11 20:23:39
	839	2026-02-06 11:43:30	90.47	2024-10-31 08:11:51
	421	2026-02-06 11:43:30	93.23	2025-02-18 22:49:44
	946	2024-02-28 15:15:26	97.09	2026-01-15 20:34:18
	507	2026-02-06 11:43:30	95.73	2024-08-15 19:35:17
	565	2025-09-01 18:44:47	91.37	2026-01-26 05:11:32
	191	2026-02-06 11:43:30	93.62	2024-01-15 12:48:43
	66	2025-02-25 22:54:51	90.8	2025-09-02 14:18:54
	705	2025-01-19 03:41:58	92.65	2024-11-13 08:23:26
	420	2024-04-06 14:29:34	94.72	2025-07-06 21:00:57
	947	2025-06-14 11:18:04	94.52	2024-08-15 00:27:16
	302	2024-07-02 22:03:30	97.58	2025-02-23 15:19:01
	724	2026-02-06 11:43:30	91.71	2024-10-26 14:39:13
	637	2026-02-06 11:43:30	97.44	2025-11-15 14:44:00

5. Vistas

Vista sobre la consulta 6,

```

create view vista_limpieza_urgente as
select numero, tipo, planta, estado
from HABITACION
where estado in ('Sucia', 'Mantenimiento');

select * from vista_limpieza_urgente vlu;

```

mpieza_urgente 1 X

from vista_limpieza_urgente vlu | Enter a SQL expression to filter results (use Ctrl

AZ numero	AZ tipo	AZ planta	AZ estado
441	Suite	3	Sucia
343	Doble	1	Mantenimiento
188	Familiar	4	Sucia
454	Deluxe	4	Mantenimiento
248	Doble	5	Sucia
429	Deluxe	1	Sucia
107	Deluxe	4	Mantenimiento
327	Simple	1	Mantenimiento
425	Simple	2	Sucia
212	Doble	1	Mantenimiento
300	Simple	2	Mantenimiento
182	Simple	2	Sucia
443	Deluxe	4	Mantenimiento
352	Doble	3	Sucia
272	Familiar	5	Sucia
241	Suite	3	Mantenimiento
433	Suite	1	Sucia
176	Doble	1	Sucia
331	Familiar	4	Sucia
201	Doble	4	Mantenimiento
281	Doble	4	Mantenimiento
130	Doble	2	Mantenimiento

Vista sobre la consulta 7,

```

CREATE VIEW vista_pagos_vip AS
SELECT f.idFACTURA, f.fecha_emision, p.cantidad
FROM FACTURA f
INNER JOIN PAGO p ON f.idFACTURA = p.FACTURA_idFACTURA
WHERE p.cantidad > 90;

select * from vista_pagos_vip vpv;

```

pagos_vip 1 X

* from vista_pagos_vip vpv Enter a SQL expression to filter results (use Ctrl+Sp)

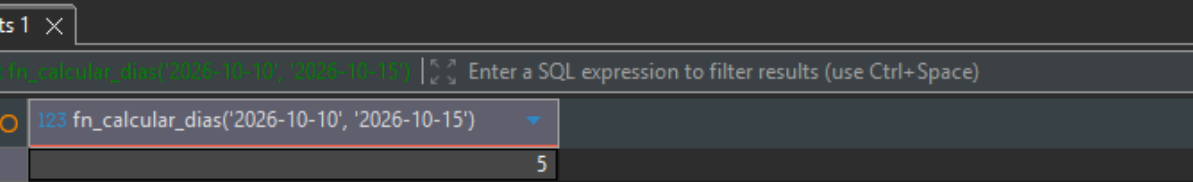
idFACTURA	fecha_emision	cantidad
917	2026-02-06 11:43:30	99.96
579	2026-02-06 11:43:30	93.52
851	2026-02-06 11:43:30	92.3
151	2025-03-25 23:46:49	95.21
745	2026-01-16 20:22:47	94.94
126	2025-08-03 20:00:10	97.93
494	2026-02-06 11:43:30	95.76
839	2026-02-06 11:43:30	90.47
421	2026-02-06 11:43:30	93.23
946	2024-02-28 15:15:26	97.09
507	2026-02-06 11:43:30	95.73
565	2025-09-01 18:44:47	91.37
191	2026-02-06 11:43:30	93.62
66	2025-02-25 22:54:51	90.8
705	2025-01-19 03:41:58	92.65
420	2024-04-06 14:29:34	94.72
947	2025-06-14 11:18:04	94.52
302	2024-07-02 22:03:30	97.58
724	2026-02-06 11:43:30	91.71
637	2026-02-06 11:43:30	97.44
856	2025-09-29 07:53:39	97.15

FUNCIONES

Calcular los días de una reserva.

```
delimiter //
-- create function fn_calcular_dias(p_fecha_entrada datetime, p_fecha_salida datetime)
-- returns int
-- deterministic
-- begin
--     if p_fecha_entrada >= p_fecha_salida then
--         signal sqlstate '45000'
--         set message_text = 'error: la fecha de entrada debe ser menor a la fecha de salida.';
--     end if;
--
--     return datediff(p_fecha_salida, p_fecha_entrada);
-- end //
delimiter ;

select fn_calcular_dias('2026-10-10', '2026-10-15');
```



The screenshot shows a SQL IDE interface. The top pane contains the SQL code for creating the function `fn_calcular_dias` and executing it. The bottom pane shows the results of the execution. The function is called with the arguments `'2026-10-10'` and `'2026-10-15'`, and the result is `5`.

fn_calcular_dias('2026-10-10', '2026-10-15')
5

Calcular el total pagado de una factura

```
delimiter //
create function fn_total_pagado_factura(p_idfactura int)
returns decimal(10,2)
deterministic
begin
    declare v_total decimal(10,2);
    declare v_existe int;

    -- comprobar si la factura existe
    select count(*) into v_existe from FACTURA f where idfactura = p_idfactura;

    if v_existe = 0 then
        signal sqlstate '45000'
        set message_text = 'error: la factura indicada no existe en la base de datos.';
    end if;

    -- calcular el total
    select ifnull(sum(cantidad), 0) into v_total
    from PAGO p
    where factura_idfactura = p_idfactura;

    return v_total;
end //
delimiter ;

select fn_total_pagado_factura(1);
```

123 fn_total_pagado_factura(1) 177.99

PROCEDIMIENTOS

Registra un nuevo cliente de una manera comoda

```
delimiter //
create procedure sp_crear_cliente(
  in p_dni varchar(9),
  in p_nombre varchar(20),
  in p_apellidos varchar(40),
  in p_telefono varchar(12),
  in p_email varchar(30),
  in p_direccion varchar(45)
)
begin
  insert into CLIENTE (dni, nombre, apellidos, telefono, email, direccion)
  values (p_dni, p_nombre, p_apellidos, p_telefono, p_email, p_direccion);
end //
delimiter ;

call sp_crear_cliente("12345679E", "Saul", "Rodriguez", "34642098240", "saul@gmail.com", "BOLLULLOS CITY");
```

	Value
Rows	1
Time	0.125s
	Mon Mar 23 18:51:31 CET 2026
	Mon Mar 23 18:51:31 CET 2026
	call sp_crear_cliente("12345679E", "Saul", "Rodriguez", "34642098240", "saul@gmail.com", "BOLLULLOS CITY")

Mostrar el resumen de una factura

```
delimiter //
create procedure sp_mostrar_total_factura(in p_idfactura int)
begin
    select idfactura,
           fecha_emision,
           fn_total_pagado_factura(p_idfactura) as total_pagado
    from FACTURA
    where idfactura = p_idfactura;
end //
delimiter ;

call sp_mostrar_total_factura(1);
```

FACTURA 1 × Statistics 1

sp_mostrar_total_factura(1) | Enter a SQL expression to filter results (use Ctrl+Space)

123 idfactura	fecha_emision	123 total_pagado
1	2026-02-06 11:43:30	177.99

Cancelar una reserva

```
delimiter //
● create procedure sp_cancelar_reserva(in p_idreserva int)
begin
  update RESERVA
  set estado = 'cancelada'
  where idreserva = p_idreserva;
end //
delimiter ;

call sp_cancelar_reserva(15);

● select *
  from RESERVA r
 where r.idRESERVA = 15;
```

RVA 1 X

123 idRESERVA 15 | 93225626A EMPLEADO_DNI | 2025-04-04 20:12:50 fecha_entrada | 2025-04-07 20:12:50 fecha_salida | Cancelada estado | 82911840A CLIENTE_DNI

idRESERVA	EMPLEADO_DNI	fecha_entrada	fecha_salida	estado	CLIENTE_DNI
15	93225626A	2025-04-04 20:12:50	2025-04-07 20:12:50	Cancelada	82911840A

TRIGGERS

Verificar que los pagos que se añaden son positivos

```
delimiter //
● create trigger trg_verificar_pago_positivo before insert on PAGO
  for each row
  begin
    if new.cantidad <= 0 then
      signal sqlstate '45000'
      set message_text = 'error: la cantidad del pago debe ser mayor que cero.';
    end if;
  end //
delimiter ;
```

```
● insert into PAGO (idpago, fecha, cantidad, factura_idfactura)
  values (9999, now(), -50.00, 1);
```

Its 1 ×

Insert into PAGO (idpago, fecha, cantidad, factura_idfactura) values (9999, now(), -50.00, 1); Enter a SQL expression to filter results (use Ctrl+F)

SQL Error [1644] [45000]: error: la cantidad del pago debe ser mayor que cero.

Hace que la habitación de la cual acaba de salir se marque sola como sucia.

```
delimiter //
● create trigger trg_habitacion_sucia after update on RESERVA
  for each row
  begin
    if new.estado = 'check-out' and old.estado != 'check-out' then
      update habitacion
      set estado = 'sucia'
      where idhabitacion in (
        select habitacion_idhabitacion
        from reserva_habitacion
        where reserva_idreserva = new.idreserva
      );
    end if;
  end //
delimiter ;
```

Probamos primero.

```
select r.idreserva, r.estado as estado_reserva, h.idhabitacion, h.estado as estado_habitacion
from RESERVA r
inner join RESERVA_HABITACION rh on r.idreserva = rh.reserva_idreserva
inner join HABITACION h on rh.habitacion_idhabitacion = h.idhabitacion
where r.idreserva = 5;
```

A(+) 1 X

idreserva | estado as estado_reserva | idhabitacion | h.estado as estado_habitacion | Enter a SQL expression to filter results (use Ctrl+Space)

idreserva	estado_reserva	idhabitacion	estado_habitacion
5	Check-in	9	Ocupada

Ahora lo cambiamos

```
select r.idreserva, r.estado as estado_reserva, h.idhabitacion, h.estado as estado_hab
from RESERVA r
inner join RESERVA_HABITACION rh on r.idreserva = rh.reserva_idreserva
inner join HABITACION h on rh.habitacion_idhabitacion = h.idhabitacion
where r.idreserva = 5;

update RESERVA
set estado = 'check-out'
where idreserva = 5;
```

VA(+) 1 X

idreserva, estado as estado_reserva, idhabitacion, estado as estado_hab | Enter a SQL expression to filter results (use Ctrl-)

idreserva	estado_reserva	idhabitacion	estado_habitacion
5	Check-out	9	Sucia

Valoración personal

Esta tercera entrega ha sido más fácil y tranquila simplemente escribir algo de código y que funcione bien en mi opinión la mejor pq los problemas eran solo cosas del código y no tema de claves primarias y eso.

<https://github.com/govedev/RoomKeyDB>